

Entwicklung eines kryptografisch sicheren und modernen Messenger-Dienstes

unter eigenständiger Implementierung eines TLS-ähnlichen Protokolls

Anton Aparin

Ausgangslage & Fragestellung

Idee und Vorgehensweise

Entwicklung der Chat-App

Ergebnisse & Sicherheitsanalyse

Fazit und Ausblick

Idee & Ziel



- HTTPS wird von allen verwendet.
- Nur von sehr wenigen verstanden.
- Auch für Software-Entwickler meistens eine „Blackbox“.
- **Kann ich auch selbst ein sicheres Protokoll für eine Messenger-App implementieren?**

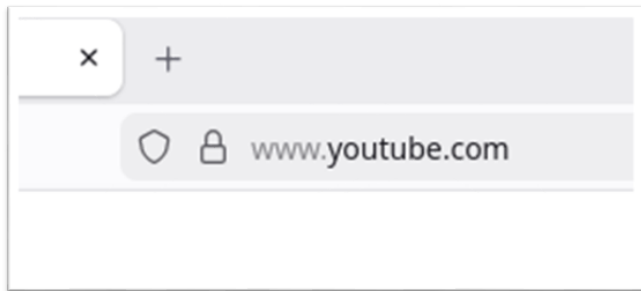


Abbildung 1: Schlosssymbol in Firefox



1. **Theorie:** Analyse von TLS und Algorithmen.
 2. **Design:** Entwurf der Anwendung, verwendete Technologien, Architektur.
 3. **Implementierung:** Alle Komponenten wurden programmiert.
 4. **Test:** Pen-Testing durchgeführt, um Sicherheit der Anwendung zu prüfen.
- **Hauptquellen:**
 - Network Security: Private Communication in a Public World, C. Kaufman
 - Cloudflare
 - Auch viel **eigene Erfahrung** im Programmieren.

- TLS = Modernes Protokoll, Standard für sichere Datenübertragung¹
- HTTP + TLS = HTTPS



¹vgl. C. Kaufman, R. Perlman, M. Speciner, und R. Perlner [1], Cloudflare [2]

Theorie (ii)

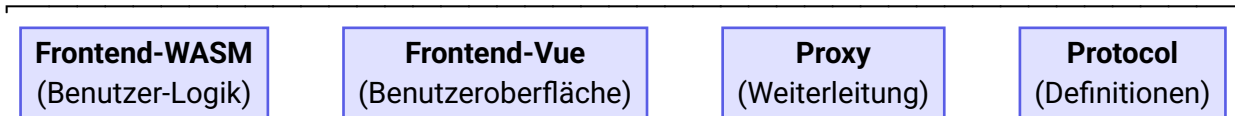


- Die 4 Sicherheitskonzepte von TLS sind:
 1. **Vertraulichkeit**: Nur Sender und Empfänger lesen mit.
 2. **Integrität**: Nachrichten können unbemerkt nicht verändert werden.
 3. **Authentizität**: Sicherstellen, dass der Partner der ist, für den er sich ausgibt.
 4. **Forward Secrecy**: Auch wenn ein Langzeitschlüssel gestohlen wird, bleiben alte Nachrichten sicher.

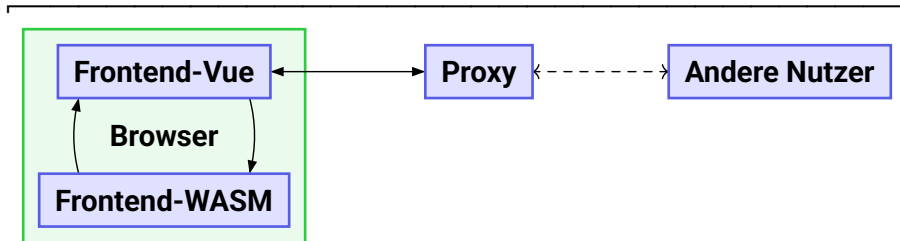
Architektur & Design



Die 4 Komponenten der Anwendung



Datenfluss der Anwendung



Verwendete Technologien: **Rust** (Programmiersprache), **Vue** (JavaScript-Framework), **WASM** (binäres Instruktionsformat), **JavaScript** (Skriptsprache)

Implementierung & Hürden



- Zunächst die wichtigsten Teile von `protocol` implementiert.
- Danach an allen Teilen gleichzeitig weitergearbeitet.

3 mögliche Zustände eines Chats (Zustandsmaschine)



- Alle Fehler müssen behoben werden.
- **Rust** war aufgrund von gutem Error-Handling von großem Vorteil.
- Auch **State-Mismatches** müssen genau definiert sein.

Codebeispiel



```
let message = match packet.message.decrypt(cipher, packet.nonce) {  
  Ok(m) => m,  
  Err(e) => {  
    message_updates.push(MessageUpdateToFrontend {  
      chat_id: packet.id.to_string(),  
      message: Message::System(format!(  
        "{} attempted to send message but decryption failed: {}",  
        to_hex(&packet.sender_ver_key.to_bytes()),  
        e  
      )),  
    });  
    return Ok((message_updates, None));  
  }  
};
```

Transparenz durch Versionskontrolle



- **Git & GitHub**
- **Nachvollziehbarkeit:** 30 Commits dokumentieren den gesamten Entwicklungsprozess.
- **Open Source:** Der vollständige Quellcode und die Dokumentation sind öffentlich zugänglich.
- **Fehlermanagement:** Systematische Behebung von Bugs durch Versionshistorie.

Commits on Oct 31, 2025

chore(frontend-vue): use alert() instead of console.log() Antosser committed on Oct 31, 2025	Verified 37ab492	
fix(frontend-vue): Make chat list fancier Antosser committed on Oct 31, 2025	Verified 6b355eb	
fix(frontend-vue): Display messages as text instead of json Antosser committed on Oct 31, 2025	Verified e70c303	
feat: Add docker support Antosser committed on Oct 31, 2025	Verified 0585eaa	
chore(frontend-vue): Only display the last 4 digits of the chat ID Antosser committed on Oct 31, 2025	Verified 3177cfd	

Abbildung 2: GitHub Commits vom 31. Oktober 2025



- **Werkzeug:** Burp-Suite als Man-in-the-Middle-Proxy.
- **Test-Szenarien & Ergebnisse:**
 - ▶ **Manipulation:** Gezielte Bit-Veränderung wurde durch Integritätsprüfung (AEAD) sofort erkannt.
 - ▶ **Replay-Attacks:** Verdoppelte Pakete wurden abgelehnt.
 - ▶ **Daten-Einsicht:** Trotz vollem Zugriff auf den Datenstrom blieb der Inhalt dank E2E-Verschlüsselung unlesbar.

Ergebnisse



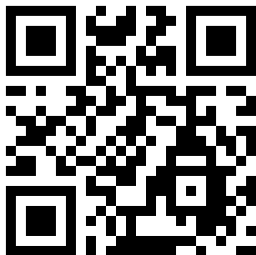
- Quellcode, die Arbeit und eine Demo sind veröffentlicht unter <https://aba.antonaparin.com>
- Anwendung funktioniert, keine Sicherheitslücken wurden entdeckt.
- Die definierten Sicherheitspunkte wurden erfolgreich erreicht.
 1. **Vertraulichkeit:** Die Nachrichten können auch vom Proxy nicht entschlüsselt werden.
 2. **Integrität:** Jede Änderung wird sofort erkannt.
 3. **Authentizität:** Nach dem Handshake kann niemand imitiert werden.
 4. **Forward Secrecy:** Wenn zukünftig Schlüssel gestohlen werden, können Nachrichten nicht entschlüsselt werden.

Abschluss & Reflexion



- „Security by Design“ statt „Security by Obscurity“
- Prozess war von unerwarteten Fehlern geprägt
- Tieferes Verständnis der Technologien
- Ausblick:
 - Gruppenchats
 - Einfacheres teilen von öffentlichen Schlüsseln
 - Horizontale Skalierung
 - Wiederherstellbare Chats
- Leitfrage wurde beantwortet:
 - Eigenes Protokoll konnte implementiert werden

- [1] C. Kaufman, R. Perlman, M. Speciner, und R. Perlner, *Network Security: Private Communication in a Public World*, 3. Aufl. Hoboken, NJ: Pearson Education, 2022. Zugriffen: 5. September 2025. [Online]. Verfügbar unter: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=934860
- [2] Cloudflare, „What Is Transport Layer Security?“, Zugriffen: 25. September 2025. [Online]. Verfügbar unter: <https://www.cloudflare.com/learning/ssl/transport-layer-security-tls/>



<https://aba.antonaparin.com>